The Transformer Architecture

The Engine Behind Large Language Models

A simplified look at how LLMs process text

KavinSchool

Before Transformers: The Sequential Era

Pre-2017 (RNNs, LSTMs):

- Processed text **one word at a time**
- Slow to train, struggled to remember early context in long sentences

> **Terminology: RNN** (Recurrent Neural Network) and **LSTM** (Long Short-Term Memory) are older sequence models. Because they process data step-by-step, they become a bottleneck for large-scale training.

The Transformer Breakthrough

Transformers (2017):

- Introduced by Google in "Attention Is All You Need"
- Process **all words simultaneously** using **self-attention**
- Enabled massive scaling and huge context windows

How It Works: Steps 1 & 2

1. Tokenization: Breaking text into manageable chunks

```
"The cat sat on the mat"
→ [The, cat, sat, on, the, mat]
→ [1234, 5678, 9012, 3456, 1234, 7890]
```

2. Embedding: Turning tokens into vectors that capture meaning

```
"cat"   → [0.12, -0.34, 0.56, ...] (768 dimensions in GPT-2)
"dog"   → [0.11, -0.30, 0.52, ...] (Nearby, similar meaning!)
"table" → [0.89,  0.23, -0.67, ...] (Far away, different meaning)
```

How It Works: Step 3

3. Self-Attention

For each word, the model asks: **"Which other words should I pay attention to?"**

> "The **bank** was closed because it was a **holiday**."

- The word "bank" attends strongly to "closed" and "holiday"
- The model concludes this is a **financial institution** (not a riverbank)

> Transformers understand relationships between words, regardless of their distance in the sequence.

How It Works: Steps 4-6

4. Feed-Forward Networks: After attention, each position passes through a neural network to transform the representation.

5. Repeat: Models stack many Transformer blocks on top of each other.

- GPT-4 has **120+ layers**. Each layer refines the understanding.

6. Predict Next Token: The final layer outputs a **probability distribution** over all possible next tokens.

```
"The cat sat on the" → {mat: 12%, floor: 8%, roof: 5%, couch: 4%, ...}
```

> The model samples from this distribution (controlled by temperature).

The Cost of Attention

Key Insight: Attention is O(n²)

Self-attention compares **every token to every other token**.

For a sequence of length n:

- It must compute n^2 attention scores
- This is why context windows have limits and are computationally expensive

> A 128K token context = 128,000² = **16.4 billion** attention computations!

Parameters = Knowledge

Parameters are the **weights** (numbers) the model learned during training.

More parameters = more patterns learned = better performance.

Model	Parameters	Analogy
GPT-2	1.5 billion	A well-read college student
GPT-3	175 billion	An expert in many fields
GPT-4	~1.8 trillion (estimated)	A team of experts
Llama 3.1	8B / 70B / 405B	Choose your size

Pre-training vs Fine-tuning vs Prompting

Stage	What Happens	Cost	Who Does It
Pre-training	Learn language from the internet	$10M-$100M+	OpenAI, Google, Meta
Fine-tuning	Specialize on a specific domain/task	$100-$10K	Companies
RLHF	Align with human preferences	$1M+	Model providers
Prompting	Guide behavior with instructions	$0.001/call	**You! (this course)**

> This course focuses on **prompting**, the most accessible and cost-effective way to get value from LLMs.

Further Reading

Deep Dives for the Curious

KavinSchool

Resources to Learn More

- **"Attention Is All You Need" (2017):** The original Transformer paper by Google researchers
- **Jay Alammar's "The Illustrated Transformer":** The best visual explanation of the architecture on the web
- **Andrej Karpathy's "Let's build GPT":** A practical video guide to building a tiny Transformer from scratch